TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS

A LAB REPORT
ON
Genetic Algorithms in AI

SUBMITTED BY:

Name: Nabina Thapa
Roll No.: 080BCT047
Lab No.: 06

SUBMITTED TO:

Department of Electronics
and Computer Engineering

Contents

1 Objectives

1. To understand how a genetic algorithm represents a local control policy as a chromosome.
2. To model the painter robot using a lookup table based on its current square and local surroundings.
3. To evolve a population of 50 chromosomes for 200 generations in a 20 by 40 empty room.
4. To compare the evolved policy in an empty room with its behaviour in a furnished room containing 100 obstacle cells.
5. To study the effects of selection, crossover, mutation, repeated trials, and final-population visualisation.

2 Theory

A genetic algorithm searches for good solutions by maintaining a population of candidate chromosomes. Each chromosome is evaluated with a fitness function, and better candidates are more likely to contribute genes to the next generation. Crossover combines parent chromosomes, while mutation preserves variation by changing selected genes.

2.1 Chromosome and State Representation

The painter robot does not store a complete map of the room. Instead, its action is selected from a table indexed by the local state

$$[c, f, l, r],$$

where c records whether the current square is painted, and f, l, and r describe the forward, left, and right cells. The current square has two possible values and each neighbouring cell has three possible values, giving

$$2 \times 3 \times 3 \times 3 = 54$$

possible states. Therefore, one chromosome contains 54 genes. Each gene stores one of four actions: no turn, turn left, turn right, or choose a random left/right turn.

2.2 Fitness

The fitness of a chromosome is the fraction of all reachable empty cells that are painted before the safety horizon is reached:

$$F = \frac{\text{number of painted empty cells}}{\text{number of paintable cells}}.$$

The experiment uses three starting states while evolving a chromosome and eight starting states for validation. Averaging over several starts reduces the chance that a policy is selected only because of one convenient initial position.

2.3 Obstacle Layout

The furnished-room experiment uses the same 20 by 40 room with three rectangular furniture blocks. The blocks occupy exactly 100 cells, leaving 700 paintable cells. A chromosome evolved in the empty room is first transferred without retraining, and a second population is then evolved directly in the furnished room.

3 Methodology

The experiment was implemented in Python 3. The state table is generated from the Cartesian product of the four local observations, and the robot simulator returns both the coverage fraction and the complete trajectory. A fixed seed of 40731 was used so that the saved figures and summary values can be reproduced.

The independent implementation uses rank-weighted parent sampling, two-point crossover, three elite chromosomes, and a mutation rate of 0.0025. Whenever a gene mutates, it is replaced by a different action. Each population contains 50 chromosomes and is evolved for 200 generations.

Parameter	Value
Population size	50 chromosomes
Chromosome length	54 genes
Generations	200
Training starts	3 per chromosome
Validation starts	8
Selection	Rank-weighted sampling with 3 elites
Crossover	Two-point crossover
Mutation probability	0.0025 per gene

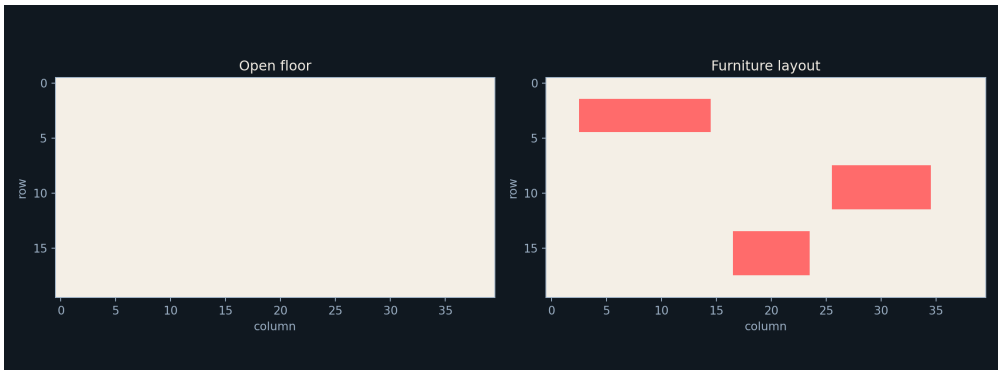

Figure 1: Empty room and the 100-cell furnished-room layout.

4 Implementation

4.1 Simulation and Genetic Operators

The file `code/redone_painter_ga.py` contains the simulator, genetic operators, validation routines, and plotting functions. The robot paints its current cell, reads the four

local observations, applies the action stored at the corresponding gene, and attempts to move forward. Invalid moves are counted in the trajectory but do not paint a new cell.

The main command used to generate the experiment is:

```
python code/redone_painter_ga.py --output outputs
```

The output directory contains the room layouts, generation curves, final chromosome heat maps, trajectories, CSV fitness histories, and a JSON summary of the numerical results.

4.2 Assignment 1 Strategy

Before evolution, a simple hand-designed strategy for the empty room is a boustrophedon sweep. The robot traverses one row, turns at the boundary, shifts by one row, and traverses the next row in the opposite direction. This strategy reduces repeated long walks and gives a useful reference for judging the evolved local policy.

4.3 Assignment 3 Innovations

Two local innovations that can produce a sudden increase in fitness are:

1. A forward-block rule that turns toward an open side when the forward cell is a wall or furniture.
2. A painted-neighbour rule that treats an already painted strip as a boundary and directs the robot toward the adjacent unpainted strip.

Both innovations convert local observations into approximate memory of the route without adding an explicit global map.

5 Results

5.1 Empty Room

The best empty-room chromosome reached complete coverage for one validation start and achieved a mean coverage of 97.77% over eight validation starts. The weakest validation start covered 88.00% of the 800 paintable cells.

Table 1: Empty-room result summary

Measure	**Result**
Paintable cells	800
Mean coverage	97.77%
Best validation run	100.00%
Weakest validation run	88.00%
Mean steps	2096.75

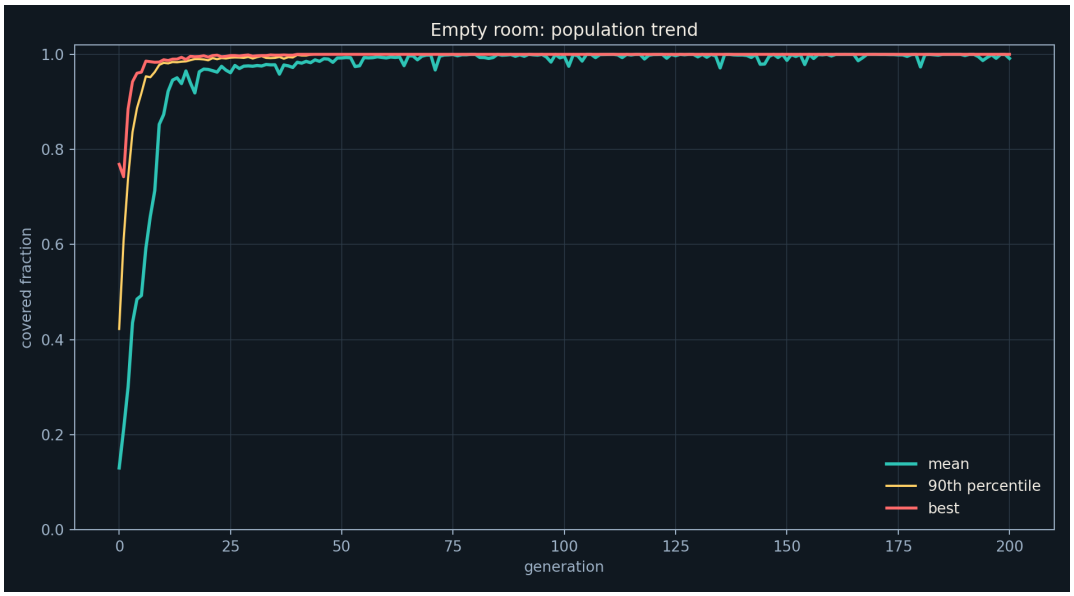

Figure 2: Mean, 90th-percentile, and best fitness across empty-room generations.

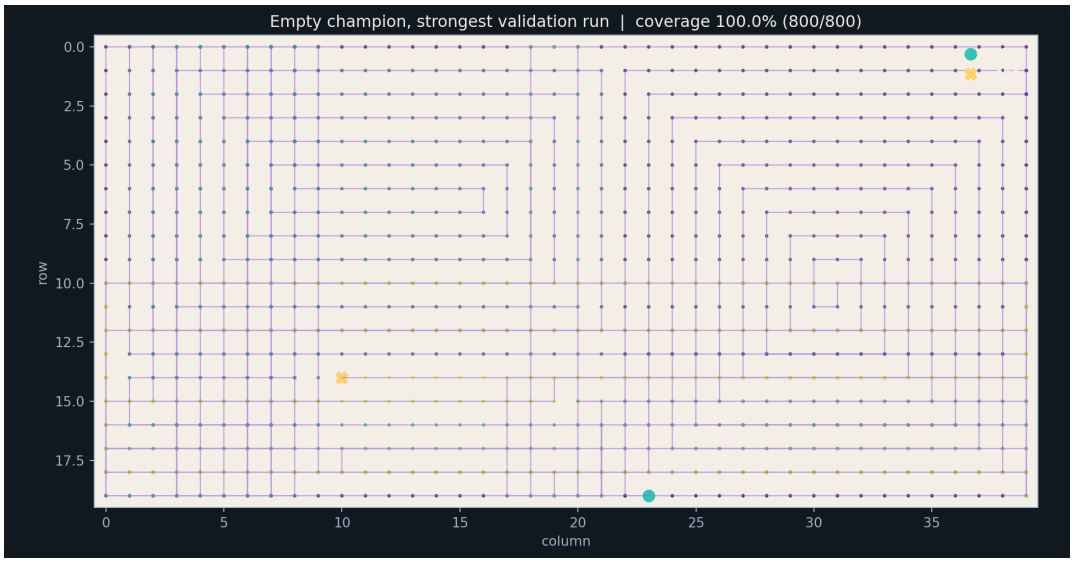

Figure 3: Trajectory of the strongest empty-room validation run.

5.2 Final Population

The final chromosome heat map shows the action selected for every state in all 50 chromosomes. Repeated vertical bands indicate states where selection converged toward a common action, while irregular bands indicate states that remain sensitive to the route history.

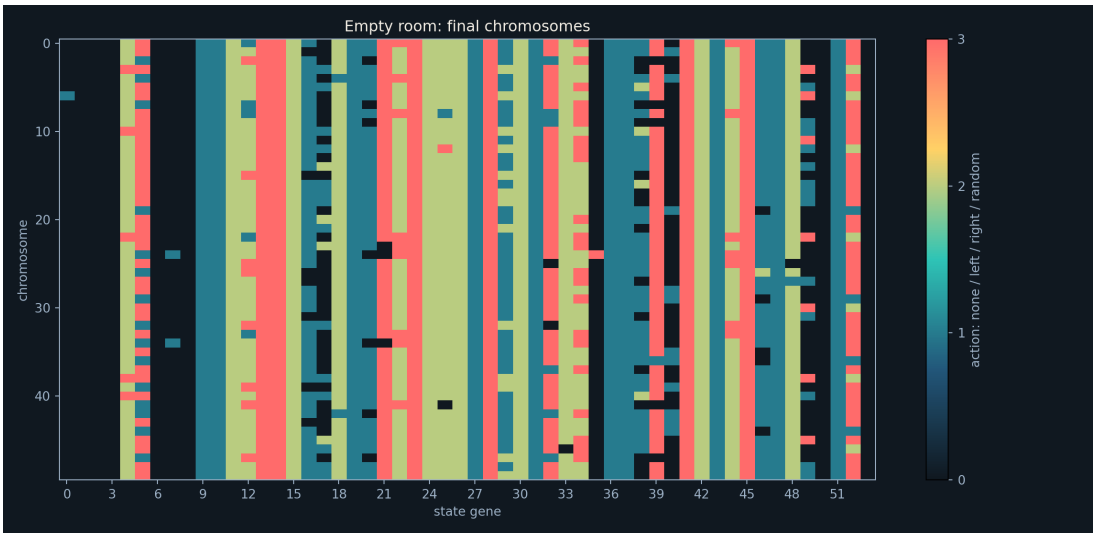

Figure 4: Final population after evolution in the empty room.

5.3 Assignment 4: Furniture Transfer

The empty-room champion was tested in the furnished room without retraining. It achieved 96.84% mean coverage over 700 paintable cells. A new population evolved directly in the furnished room achieved 97.36% mean coverage.

Table 2: Transfer and furnished-room comparison

Test	Mean	Best	Weakest	Cells
Empty champion transferred	96.84%	100.00%	91.71%	700
Furnished room evolved directly	97.36%	100.00%	84.29%	700

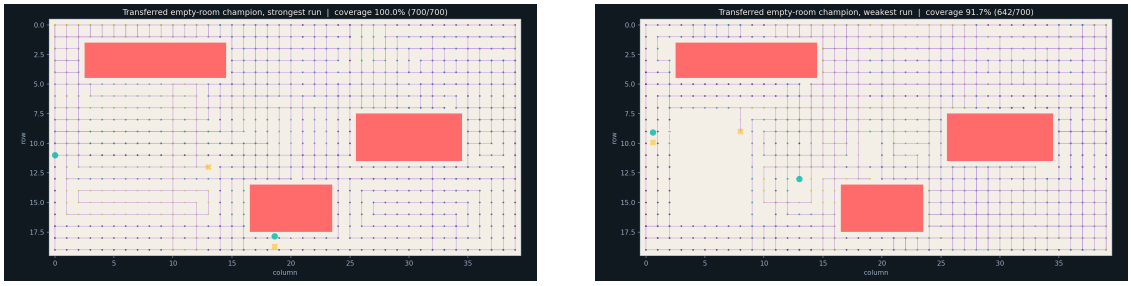

Figure 5: Strongest and weakest transfer trajectories in the furnished room.

Figure 6: Fitness history during evolution in the furnished room.

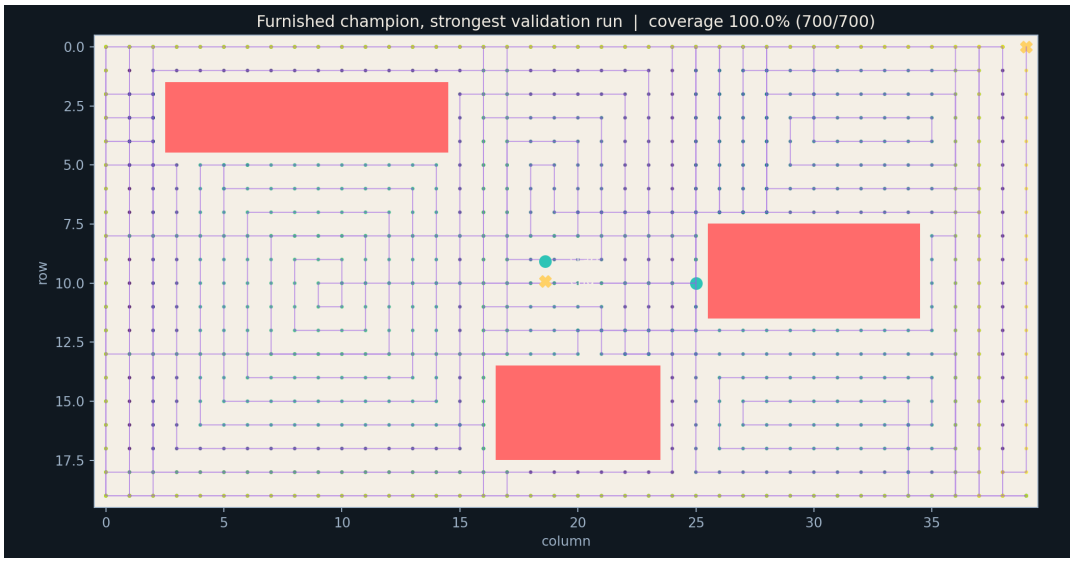

Figure 7: Strongest validation trajectory for the furnished-room champion.

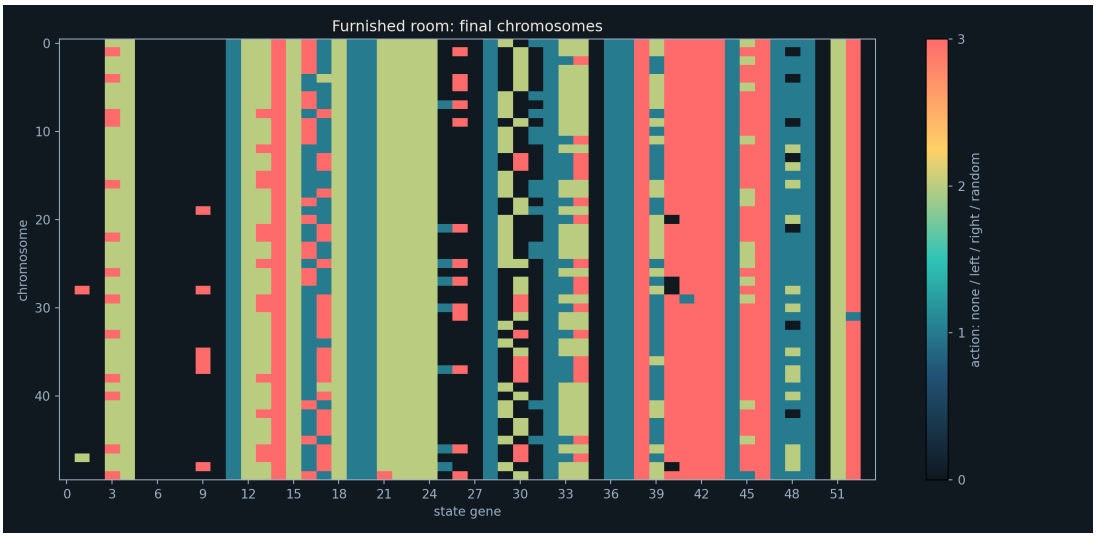

Figure 8: Final chromosome population after furnished-room evolution.

6 Discussion

The fitness curves rise rapidly during the first generations and then stabilise near complete coverage. The best curve reaches a high value earlier than the mean curve because one successful local rule combination can appear before crossover spreads it through the rest of the population.

The empty-room chromosome transfers well to the furnished layout because several of its actions respond to local boundaries. However, furniture changes the meaning of a useful turn: a rule that is helpful at an outside wall may cause a detour near a rectangular obstacle. This explains why the transferred champion has a lower mean than its empty-room result and why the directly evolved furnished-room population is more obstacle-specific.

The results also show that the highest single validation score is not enough to describe a policy. Both evolved champions achieved 100.00% on one starting state, but their weakest validation runs were lower. Repeated starts therefore provide a more reliable measure of general behaviour.

7 Conclusion

This lab demonstrated how a genetic algorithm can evolve a useful painter-robot policy from local observations. A chromosome of only 54 action genes was evaluated, selected, recombined, and mutated over 200 generations. In the alternate run, the empty-room champion averaged 97.77% coverage, transferred to the furnished layout with 96.84% coverage, and direct furnished-room evolution averaged 97.36%.

The experiment confirms that genetic algorithms can discover compact control rules without explicitly programming a global route. At the same time, the furniture experiment shows that performance depends on the environment and the starting state. Selection, crossover, mutation, repeated validation, and trajectory plots together provide a practical way to analyse the quality and limitations of the evolved policy.